



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
 ESCUELA DE INGENIERÍA
 DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2223 — Teoría de Autómatas y Lenguajes Formales — 2° 2023

PAUTA EXAMEN

Pregunta 1

Una gramática $\mathcal{G} = (V, \Sigma, P, S)$ se dice $\text{LL}(k)$ ssi para todas dos reglas distintas $Y \rightarrow \gamma_1, Y \rightarrow \gamma_2$ y para todo $S \xrightarrow[\text{lm}]{*} uY\beta$ se tiene que:

$$\text{first}_k(\gamma_1\beta) \cap \text{first}_k(\gamma_2\beta) = \emptyset$$

Una gramática $\mathcal{G} = (V, \Sigma, P, S)$ se dice $\text{LL}(k)$ -fuerte ssi para todas dos reglas distintas $Y \rightarrow \gamma_1, Y \rightarrow \gamma_2$ se tiene que:

$$\text{first}_k(\gamma_1) \odot_k \text{follow}_k(Y) \cap \text{first}_k(\gamma_2) \odot_k \text{follow}_k(Y) = \emptyset$$

Pregunta 1.1

En esta pregunta nos piden demostrar que si $k = 1$, entonces $\mathcal{G}$ es $\text{LL}(k)$ -fuerte. Sea $\mathcal{G}$ es una gramática libre de contexto $\text{LL}(k)$ y $Y \rightarrow \gamma_1, Y \rightarrow \gamma_2$ dos reglas distintas. Como sabemos que $\mathcal{G}$ es $\text{LL}(1)$, entonces sabemos que para todo $S \xrightarrow[\text{lm}]{*} uY\beta$, $\text{first}_1(\gamma_1\beta) \cap \text{first}_1(\gamma_2\beta) = \emptyset$. Veremos 3 casos:

1. $\varepsilon \notin \text{first}_1(\gamma_1)$ y $\varepsilon \notin \text{first}_1(\gamma_2)$. Luego, para todo $S \xrightarrow[\text{lm}]{*} uY\beta$, tenemos que $\text{first}_1(\gamma_1\beta) \cap \text{first}_1(\gamma_2\beta) = \text{first}_1(\gamma_1) \cap \text{first}_1(\gamma_2) = \emptyset$. Luego, $\text{first}_1(\gamma_1) \odot_1 \text{follow}_1(Y) \cap \text{first}_1(\gamma_2) \odot_1 \text{follow}_1(Y) = \text{first}_1(\gamma_1) \cap \text{first}_1(\gamma_2) = \emptyset$.
2. $\text{SPDG } \varepsilon \in \text{first}_1(\gamma_1)$ y $\varepsilon \notin \text{first}_1(\gamma_2)$. Luego, para todo $S \xrightarrow[\text{lm}]{*} uY\beta$, tenemos que $\text{first}_1(\gamma_1\beta) \cap \text{first}_1(\gamma_2\beta) = \text{first}_1(\gamma_1\beta) \cap \text{first}_1(\gamma_2) = \text{first}_1(\gamma_1\beta) \cap \text{first}_1(\gamma_2\beta') = \emptyset$ para todo $\beta' \in (V \cup \Sigma)^*$. Por lo tanto, $\text{first}_1(\gamma_1) \odot_1 \text{follow}_1(Y) \cap \text{first}_1(\gamma_2) \odot_1 \text{follow}_1(Y) = \bigcup_{S \xrightarrow[\text{lm}]{*} uY\beta} \text{first}_1(\gamma_1\beta) \cap \bigcup_{S \xrightarrow[\text{lm}]{*} vY\beta'} \text{first}_1(\gamma_2\beta') = \emptyset$.
3. $\varepsilon \in \text{first}_1(\gamma_1)$ y $\varepsilon \in \text{first}_1(\gamma_2)$. En este caso, para todo $S \xrightarrow[\text{lm}]{*} uY\beta$, tenemos que $\text{first}_1(\beta) \subseteq \text{first}_1(\gamma_1\beta)$ y $\text{first}_1(\beta) \subseteq \text{first}_1(\gamma_2\beta)$, luego $\text{first}_1(\beta) \cap \text{first}_1(\beta) = \text{first}_1(\beta) = \emptyset$. Es por esto que $\text{first}_1(\gamma_1\beta) \cap \text{first}_1(\gamma_2\beta) = \text{first}_1(\gamma_1) \cap \text{first}_1(\gamma_2)$. Como $\varepsilon \in \text{first}_1(\gamma_1) \cap \text{first}_1(\gamma_2) \neq \emptyset$, entonces $\mathcal{G}$ no es $\text{LL}(1)$, por lo que llegamos a una contradicción y este caso no es posible.

Dicho esto, como para todos los casos $\text{first}_1(\gamma_1) \odot_1 \text{follow}_1(Y) \cap \text{first}_1(\gamma_2) \odot_1 \text{follow}_1(Y) = \emptyset$, entonces concluimos que $\mathcal{G}$ es $\text{LL}(1)$ -fuerte

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(3 puntos)** Por el caso 1, es decir, $\varepsilon \notin \text{first}_1(\gamma_1)$ y $\varepsilon \notin \text{first}_1(\gamma_2)$.
 - **(1.5 puntos)** por notar que $\text{first}_1(\gamma_1) \cap \text{first}_1(\gamma_2) = \emptyset$
 - **(1.5 puntos)** por concluir que $\text{first}_1(\gamma_1) \odot_1 \text{follow}_1(Y) \cap \text{first}_1(\gamma_2) \odot_1 \text{follow}_1(Y) = \emptyset$

- **(3 puntos)** Por el caso 2, es decir, $\varepsilon \in \text{first}_1(\gamma_1)$ y $\varepsilon \notin \text{first}_1(\gamma_2)$
 - **(1.5 puntos)** por notar que $\text{first}_1(\gamma_1\beta) \odot_1 \text{first}_1(\gamma_2\beta') = \emptyset$
 - **(1.5 puntos)** por concluir que $\text{first}_1(\gamma_1) \odot_1 \text{follow}_1(Y) \cap \text{first}_1(\gamma_2) \odot_1 \text{follow}_1(Y) = \emptyset$

Pregunta 1.2

En esta pregunta nos piden demostrar que si $k > 1$, entonces $\mathcal{G}$ no es necesariamente $\text{LL}(k)$ fuerte. Para esto, definiremos una gramática $\mathcal{G}$ que sea $\text{LL}(k)$ para todo $k \geq 2$, pero no sea $\text{LL}(k)$ -fuerte para ningún k . Sea la gramática $\mathcal{G} = (V, \Sigma, P, S)$ definida por $V = \{S, X\}$, $\Sigma = \{a, b\}$ y las siguientes reglas en P :

$$\begin{aligned} S &\rightarrow aXaa \mid bXba \\ X &\rightarrow b \mid \varepsilon \end{aligned}$$

Primero demostraremos que esta gramática es $\text{LL}(2)$.

- Si tomamos las reglas $X \rightarrow b$ y $X \rightarrow \varepsilon$, y la regla $S \xrightarrow[\text{lm}]{*} aXaa$, entonces $\text{first}_2(baa) \cap \text{first}_2(aa) = \emptyset$.
- Si tomamos las reglas $X \rightarrow b$ y $X \rightarrow \varepsilon$, y la regla $S \xrightarrow[\text{lm}]{*} bXba$, entonces $\text{first}_2(bba) \cap \text{first}_2(ba) = \emptyset$.

Como no hay más casos, $\mathcal{G}$ es $\text{LL}(2)$.

Ahora demostraremos que $\mathcal{G}$ no es $\text{LL}(2)$ -fuerte. Si tomamos las reglas $X \rightarrow b$ y $X \rightarrow \varepsilon$, tenemos que $\text{follow}_k(X) = \{aa, ba\}$, y que $\text{first}_2(b) \odot_2 \text{follow}_2(X) \cap \text{first}_2(\varepsilon) \odot_2 \text{follow}_2(X) = \{b\} \odot_2 \{aa, ba\} \cap \{\varepsilon\} \odot_2 \{aa, ba\} = \{ba, ba\} \cap \{aa, ba\} = \{ba\}$ que no es $\text{LL}(2)$ -fuerte.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(3 puntos)** Por demostrar una gramática que sea $\text{LL}(k)$ para algún k
 - **(1.5 puntos)** Por construir una gramática que sea $\text{LL}(k)$
 - **(1.5 puntos)** Por demostrar que esa gramática es $\text{LL}(k)$
- **(3 puntos)** Por demostrar que la gramática no es $\text{LL}(k)$ -fuerte
 - **(1.5 puntos)** Por calcular $\text{follow}_k(Y)$ para algún $Y \in V$
 - **(1.5 puntos)** Por demostrar que $\text{first}_k(\gamma_1) \odot_k \text{follow}_k(Y) \cap \text{first}_k(\gamma_2) \odot_k \text{follow}_1(Y) \neq \emptyset$ para algún par de reglas $Y \rightarrow \gamma_1$ y $Y \rightarrow \gamma_2$

Pregunta 2

Pregunta 2.1

Podemos demostrar que el lenguaje A no es libre de contexto usando lema de bombeo. Sea $z = a^N b^{N+1} c^{N+1} \in A$. Tomando toda descomposición tenemos los siguientes casos:

1. $vwx = a^m \vee vwx = a^m$. En este caso bastará tomar $i = 2$ e inmediatamente se cumplirá que $|uv^2wx^2y|_a \geq |uv^2wx^2y|_b$ y $|uv^2wx^2y|_b > |uv^2wx^2y|_c$, respectivamente, haciendo que $uv^2wx^2y \notin A$.
2. $vwx = c^m$. En este caso bastará tomar $i = 0$ e inmediatamente se cumplirá que $|uv^0wx^0y|_c \leq |uv^0wx^0y|_b$.
3. $vwx = a^m b^n$. En este caso tendremos varias divisiones posibles:

- a) $v = a^p \wedge x = a^q \vee v = b^p \wedge x = b^q$. Estos casos son análogos a 1.
- b) $v = a^p b^q \vee x = a^p b^q$. En este caso bastará tomar $i = 2$ y la palabra bombeada perderá la forma que se necesita, haciendo que $uv^2wx^2y \notin A$.
- c) $v = a^p \wedge x = b^q$. Considerando $i = 2$, se cumplirá que $|uv^0wx^0y|_b \geq |uv^0wx^0y|_c$.
4. $vw x = b^m c^n$. Este caso es muy similar al anterior, pero no análogo. Sus divisiones son:
- a) $v = b^p \wedge x = b^q \vee v = c^p \wedge x = c^q$. Estos casos son análogos a 1 y 2, respectivamente.
- b) $v = b^p c^q \vee x = b^p c^q$. Al tomar $i = 2$, la palabra bombeada perderá la forma que se necesita, haciendo que $uv^2wx^2y \notin A$.
- c) $v = b^p \wedge x = c^q$. Considerando $i = 0$, se cumplirá que $|uv^0wx^0y|_a \geq |uv^0wx^0y|_b$.

De acuerdo a lo anterior, el puntaje será:

- **(1 punto)** Por encontrar una palabra que pertenece al lenguaje, depende de N y puede ser bombeada.
- **(3 puntos)** Por identificar los casos y subcasos necesarios:
 - **(0.5 puntos)** Por identificar el caso 1.
 - **(0.5 puntos)** Por identificar el caso 2.
 - **(0.5 puntos)** Por identificar algún subcaso análogo en 3. (3.1)
 - **(0.5 puntos)** Por identificar algún subcaso nuevo en 3. (3.2 o 3.3)
 - **(0.5 puntos)** Por identificar algún subcaso análogo en 4. (4.1)
 - **(0.5 puntos)** Por identificar algún subcaso nuevo en 4. (4.2 o 4.3)
- **(2 puntos)** Por resolver los casos y subcasos necesarios:
 - **(0.5 puntos)** Por resolver 1 y sus análogos.
 - **(0.5 puntos)** Por resolver 2 y sus análogos.
 - **(0.5 puntos)** Por resolver 3.2 y 4.2.
 - **(0.5 puntos)** Por resolver 3.3 y 4.3.

Pregunta 2.2

Podemos usar la siguiente gramática libre de contexto para definir O :

$$\begin{aligned}
 S &\rightarrow XC \mid AY \\
 X &\rightarrow aXb \mid B \\
 Y &\rightarrow bYc \mid C \\
 A &\rightarrow aA \mid a \\
 B &\rightarrow bB \mid b \\
 C &\rightarrow cC \mid c
 \end{aligned}$$

Buscamos las palabras de la forma $a^i b^j c^k$ tienen más letras b que a , o más c que b .

En la gramática, las variables A , B , C permiten generar una cantidad arbitraria de letras a , b o c respectivamente y todas generan al menos una terminal.

Por otra parte, en S se puede elegir si tomar XC para tener más b que a o AY para tener más c que b .

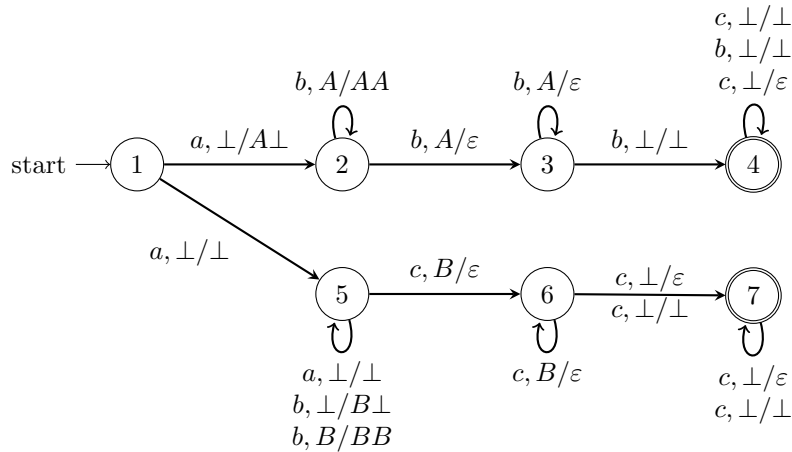
En XC , la variable C permite una cantidad arbitraria de letras c mientras que X te asegura que se genera la misma cantidad de a y b , hasta que se decida tomar la regla $X \rightarrow B$, que produce al menos una letra b al medio de la palabra, forzando más b que a . AY es análogo a XC ; A genera arbitrarios a , Y mantiene igual b y c hasta que toma C forzando a tener más c que B .

De acuerdo a lo anterior, el puntaje será:

- (1 punto) Por construir y explicar S de forma que pueda cumplir cualquiera de las dos condiciones buscadas.
- (1 punto) Por construir y explicar A , B y C correctamente.
- (1 punto) Por usar y explicar correctamente C en XC y A en AY . En otras palabras, por poder generar arbitrarias c en el caso $b > a$ y arbitrarias a en el caso $c > b$.
- (1 punto) Por construir y explicar X e Y correctamente.
 - (1 punto) Por generar más de la letra que se necesita para el caso.
 - (0.5 puntos) Por permitir generar una cantidad más de la letra que se necesita.
 - (1.5 puntos) Por explicar correctamente el funcionamiento de estas variables.

Pregunta 2.3

Podemos usar el siguiente PDA para definir O :



La idea del PDA es poder tomar de forma no determinista el camino de arriba o abajo según corresponda para la condición del “or”.

El camino de arriba corresponde al caso, $b > a$. El autómata lee las letras a y hace push de un símbolo A por cada una en el estado 2. Eventualmente, al ver la primera b , pasa al estado 3 y comienza a hacer pop de las A en el stack. Si ve una b y no quedan más A en el stack, es porque la palabra tiene más b que a y debe que aceptarla, por lo que avanza al estado final 4. El estado 4 simplemente lee las letras que quedan en la palabra y de forma no determinista saca el bottom del stack cuando ve la última c .

Similarmente, el camino de abajo resuelve el caso $b > c$, ignorando las a en el estado 5 y haciendo push de las B en este mismo estado. Cuando comienza a ver letras c avanza a 6 haciendo pop de las letras B hasta que solo queda el fondo del stack. Cuando se encuentra con este caso, de manera no determinista, remueve el bottom solo para la última letra c .

De acuerdo a lo anterior, el puntaje será:

- (1 punto) Por permitir ambas condiciones el mismo autómata usando no determinismo.
- (2.5 puntos) Por el camino superior.
 - (0.5 puntos) Por hacer y explicar correctamente el push de letras A .
 - (0.5 puntos) Por hacer y explicar correctamente el pop de letras A .

- **(0.5 puntos)** Por llegar al estado final sin vaciar el stack.
- **(0.5 puntos)** Por leer las letras c restantes sin vaciar el stack.
- **(0.5 puntos)** Por vaciar el stack correctamente.
- **(2.5 puntos)** Por el camino inferior.
 - **(0.5 puntos)** Por ignorar las letras a sin vaciar el stack.
 - **(0.5 puntos)** Por hacer y explicar correctamente el push de letras B .
 - **(0.5 puntos)** Por hacer y explicar correctamente el pop de letras B .
 - **(0.5 puntos)** Por llegar al estado final sin vaciar el stack.
 - **(0.5 puntos)** Por vaciar el stack correctamente.

Pregunta 3

Sea el autómata NFA extendido $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$, y $\delta_i \in \Delta$, tal que $\delta_i = (p, a_1 \dots a_k, p')$ con $k > 1$. Definimos un autómata un NFA tradicional $N = (Q', \Sigma, \Delta', I', F')$, tal que:

$$\begin{aligned}
Q' &= Q \uplus \left(\biguplus_i S(\delta_i) \right) \\
\Delta' &= \{ (p, a, p') \in \Delta \mid |a| = 1 \} \cup && \text{(transiciones con tamaño de lectura igual a uno)} \\
&\quad \{ (p, a_1, q_1^i) \mid \delta_i \in \Delta \wedge |k| > 1 \} \cup && \text{(transición desde un estado 'conocido' hacia el primer nuevo estado creado)} \\
&\quad \{ (q_{j-1}^i, a_j, q_j^i) \mid \delta_i \in \Delta \wedge |k| > 1 \} \cup && \text{(transiciones entre nuevos estados creados)} \\
&\quad \{ (q_{k-1}^i, a_k, p') \mid \delta_i \in \Delta \wedge |k| > 1 \} && \text{(transición de 'salida' hacia el siguiente estado 'conocido')} \\
I' &= I \\
F' &= F
\end{aligned}$$

donde $S(\delta_i) = \{q_1^i, \dots, q_{k-1}^i\}$ es el conjunto de estados definidos a partir de cada transición δ_i .

Demostraremos que $\mathcal{L}(\mathcal{A}) = \mathcal{L}(\mathcal{N})$.

$\mathcal{L}(\mathcal{A}) \subseteq \mathcal{L}(\mathcal{N})$

Sea una palabra $w \in \mathcal{L}(\mathcal{A})$, entonces existe la ejecución de aceptación:

$$\rho : p_0 \xrightarrow{u_1} p_1 \xrightarrow{u_2} \dots \xrightarrow{u_k} p_n$$

donde $p_0 \in I, p_n \in F, (p_i, u_{i+1}, p_{i+1}) \in \Delta$.

Pueden existir dos casos:

- $|u_i| = 1$.

Para este caso, el paso $p_{i-1} \xrightarrow{u_i=a_i} p_i$ no requiere de nuevos estados o transiciones.

- $|u_i| > 1$.

Sin pérdida de generalidad, para el paso $p_{i-1} \xrightarrow{u_i} p_i$ tal que $|u_i| > 1$, podemos construir las transiciones: $(p_{i-1}, a_1, q_1^j); (q_{k-1}^j, a_k, q_k^j)$, para todo $k < |u_i| - 1$; y $(q_{|u_i|-1}^j, a_{|u_i|}, p_i)$, todas pertenecientes a Δ' , por lo tanto se tiene la ejecución:

$$\rho_i : p_0 \rightarrow \dots \rightarrow p_{i-1} \rightarrow q_1^j \rightarrow \dots \rightarrow q_{|u_i|-1}^j \rightarrow p_i \xrightarrow{a_{i+1}} \dots \rightarrow p_n$$

con $p_0 \in I, p_n \in F$. Donde ρ_i representa el proceso para todo $u_i, |u_i| > 1$. Finalmente, el resultado es ρ' , el cual es una ejecución de aceptación de $\mathcal{N}$.

$\mathcal{L}(\mathcal{N}) \subseteq \mathcal{L}(\mathcal{A})$

Sea una palabra $w \in \mathcal{L}(\mathcal{N})$, entonces existe la ejecución de aceptación:

$$\rho : p_0 \xrightarrow{a_1} p_1 \xrightarrow{a_2} \dots \xrightarrow{a_n} p_n$$

donde $p_0 \in I', p_n \in F'$ y toda transición $(p_{i-1}, a_i, p_i) \in \Delta'$.

Suponga que existen estados tal que:

$$\begin{aligned} p_k &\notin Q \wedge p_j \in Q, \forall j < k \\ p_{k'} &\in Q \wedge p_j \notin Q, \forall k < j < k' \end{aligned}$$

entonces existe una transición de la forma $(p_{k-1}, a_k \dots a_{k'}, p_{k'}) \in \Delta$, por lo tanto existe una ejecución de la forma:

$$\rho_i : \xrightarrow{a_i} \dots \xrightarrow{a_{k-1}} p_{k-1} \xrightarrow{a_k \dots a_{k'}} p_{k'} \rightarrow \dots \rightarrow p_n$$

Este proceso se repite para todo $\rho_i \in \rho$ hasta que todo $p_i \in Q$. Finalmente el ρ' resultante es una ejecución de aceptación de $\mathcal{A}$ sobre w . $\square$

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(3 puntos)** Por construir el autómata $\mathcal{N}$:
 - (1 punto) Por construir los estados Q' .
 - (0.5 puntos) Por construir las transiciones con tamaño de lectura igual a uno.
 - (0.5 puntos) Por construir la transición desde un estado 'conocido' hacia el primer nuevo estado creado.
 - (0.5 puntos) Por construir las transiciones entre nuevos estados creados.
 - (0.5 puntos) Por construir la transición de 'salida' hacia el siguiente estado 'conocido'.
- **(1.5 puntos)** Por demostrar que $\mathcal{L}(\mathcal{A}) \subseteq \mathcal{L}(\mathcal{N})$.
 - (0.5 puntos) Por mencionar la ejecución $\mathcal{A}$ sobre w .
 - (0.5 puntos) Por demostrar el caso $|u_i| = 1$
 - (0.5 puntos) Por demostrar el caso $|u_i| > 1$ y concluir.
- **(1.5 puntos)** Por demostrar que $\mathcal{L}(\mathcal{N}) \subseteq \mathcal{L}(\mathcal{A})$.
 - (0.5 puntos) Por mencionar la ejecución $\mathcal{N}$ sobre w .
 - (0.5 puntos) Por explicar como encontrar los estados que difieren entre los autómatas desde las ejecuciones (p_k, p_j) .
 - (0.5 puntos) Por explicar como usar Δ' para construir la ejecución ρ' y concluir.

Pregunta 4

Algoritmo

Para un DFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ y una palabra $w = a_1 a_2 \dots a_n$, tenemos:

```
Function eval-#DFAonthefly ( $\mathcal{A}, w$ )  
   $S := [\{\}, \dots, \{\}]$  ( $n + 1$  veces)  
   $S[0] \leftarrow I$   
  for  $i = 1$  to  $n - 1$  do  
    foreach  $p \in S[i - 1]$  do  
      foreach  $(p, u, q) \in \Delta$  do  
        if  $u = a_i \dots a_{i+|u|}$  then  
           $S[i + |u|].add(q)$   
        end  
      end  
    end  
  end  
  return  $check(S[n] \cap F \neq \emptyset)$   
end
```

Explicación Correctitud

El algoritmo es similar al **NFA on-the-fly** con la modificación de que se incluye un arreglo de conjuntos en vez de solo dos conjuntos. Este arreglo guarda en cada posición los estados alcanzables al leer w

Para armar S , se itera incrementalmente por cada punto de partida de la palabra w , viendo a qué estados se ha podido llegar a ese punto, y luego se ve a qué estado se puede llegar desde ese punto usando las transiciones que cumplen para la palabra. Notar que la primera posición de S empieza con los estados iniciales del autómata

Finalmente, se revisan las letras alcanzables al leer toda la palabra revisando la intersección entre $S[n]$ y los estados finales del autómata

Explicación Complejidad

Complejidad operaciones en algoritmo

- El algoritmo tiene el arreglo S , que contiene n conjuntos de estados del autómata, por lo tanto, el iterar por los estados de una celda del arreglo de S es $\mathcal{O}(|Q|)$, y el tamaño de S es $\mathcal{O}(|Q| \cdot |w|)$. Al iniciar vacío, el construir S es $\mathcal{O}(|w|)$ y el asignar los estados iniciales a $S[0]$ es $\mathcal{O}(|Q|)$
- El algoritmo tiene tres loops anidados. El primero itera por el largo de la palabra $\mathcal{O}(|w|)$, el segundo itera por las transiciones de p , el tercero itera por los estados en S_{old} ($\mathcal{O}(|Q|)$), por la forma que está definido, sólo itera por las transiciones que parten en p , por lo que el tiempo de los últimos dos loops es $\mathcal{O}(|Q| + |\Delta|)$.
- El chequeo final ve la intersección entre dos conjuntos de tamaño $|Q|$, lo que es $\mathcal{O}(|Q|)$
- La comparación de strings es de tiempo constante, por lo que no influye la complejidad.

Con lo anterior, se tiene que la complejidad del algoritmo es

$$\mathcal{O}(|Q| + |w| + |w| \cdot (|Q| + |\Delta|))$$

Ya que $\mathcal{A} = |Q| + |\Delta|$, se tiene entonces que la complejidad es:

$$\mathcal{O}(|w| \cdot |\mathcal{A}|)$$

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(3 puntos)** Por Explicación Correctitud
 - **(1 punto)** Explica cómo se arma el conjunto S o equivalente(s) mencionando al conjunto.
 - **(1 punto)** Nota y menciona los cambios a NFA necesarios.
 - **(1 punto)** Implementa correctamente los cambios a NFA explicados.
- **(3 puntos)** Por Explicación complejidad
 - **(1 punto)** Explica la complejidad final del algoritmo ($\mathcal{O}(|w| \cdot |\mathcal{A}|)$) correctamente mencionando que se itera por la palabra y por el autómata (menciona uso de Q y Δ). Mitad del puntaje si solo menciona que porque los cambios son pocos se mantiene la complejidad de NFA-otf sin profundizar motivos y que en la práctica sea esa la complejidad.
 - **(1 punto)** Explica por qué se usa $|\Delta| + |Q|$ en vez de $|\Delta| \cdot |Q|$, mitad del puntaje si acepta que la operación es $|\Delta| \cdot |Q|$
 - **(1 punto)** Todas las iteraciones del algoritmo se ven reflejadas en la complejidad correctamente, y se menciona qué iteración corresponde a qué complejidad